PRD: Skills-first Stack Builder (working name: Brix)

Oct 25, 2026 · @Sushant Samuel

1. Overview

StackSkills is a free, open-source website where a developer picks their stack, copies one command, and lets Claude Code, Codex or Cursor build a production-ready monorepo from an empty folder.

Problem. Scaffolders such as Better-T-Stack cover a TypeScript-only slice of the stack. Starting a project with an AI agent means hand-writing rules and prompts and hunting for the right skills. Template repos explode combinatorially, so each new option multiplies maintenance.

Idea. Skills grow additively, not multiplicatively. Each option (Clerk, Neon, Razorpay, Railway and so on) maps to one or more Agent Skills (SKILL.md files). The site generates a `stack.json` spec. A CLI installs only the matching skills, writes `AGENTS.md` and a build prompt, and the agent assembles the repo by running each framework's own official scaffolder.

Who it is for. Solo builders and small teams who use AI coding agents, including developers in India who need Razorpay, Python backends or low-cost hosts that existing TS-only tools skip.

Differentiators.

- Covers Python, Fastify/Express, Razorpay, Railway/Render/Azure and queues, not only the TypeScript default stack.
- Security and a verification gate are always on, not options.
- Every popular combination is built weekly in CI and the pass/fail matrix is public.

2. Goals, non-goals, success metrics

Goals

- A user reaches a copy-ready command in under 2 minutes with no account and no backend.
- Generated repos pass typecheck, lint, tests, build and the security gate before the agent reports done.
- Works with Claude Code, Codex and Cursor from one `stack.json`, one `AGENTS.md` and a `CLAUDE.md` pointer.
- Adding a new option takes one PR to `registry.json` plus one skill, never a new template repo.
- 100% free to use; MIT-licensed core; hosted on a free static host.

Non-goals (v1)

- No hosted code generation, no accounts, no payments, no user database.
- No guarantee that every one of the thousands of option combinations works; only tested combinations are marked proven.
- Not a replacement for each vendor's own skills; we compose and pin them.

Success metrics

Metric	Target at 90 days after launch
Time from landing to copied command	under 2 min (median)
Proven presets passing weekly CI	10 of 10
Generated repos passing the verification gate on first agent run	70% or more
Options with a vendor-official or vetted skill	80% or more
Merged community PRs adding options or rules	15 or more
GitHub stars	1,000 or more

3. User flow

1. The user opens the site and picks a preset (for example SaaS starter) or starts blank.
2. They choose options across 20 sections. Incompatible options grey out with a one-line reason; dependent choices auto-correct.
3. They optionally open the Advanced accordion and type extra requirements.
4. The site shows a summary, the list of skills that will be installed (with licenses and pinned versions) and one command: `npx stackskills init --config <id>`.
5. In an empty folder, the user runs the command. The CLI asks consent for each third-party install, installs the skills for the chosen agents, and writes `stack.json`, `AGENTS.md`, `CLAUDE.md` and `PROMPT.md`.
6. The user opens their agent and pastes the one-line prompt (`Read PROMPT.md and build the project`). The agent scaffolds with official CLIs, applies skills, then runs the verification gate.
7. The agent reports a pass/fail checklist and a `NEXT_STEPS.md` of manual items (keys, DNS, dashboards).

4. Features in detail

F1. Stack builder UI

A static single-page site with 20 sections of 3 to 6 options each, a live summary panel and a copy button.

- Single-select sections by default; multi-select for AI tools used.
- Every option shows a short description and a badge: Proven (passed CI), Experimental (skill exists, not CI-tested) or Needs skill (hidden until a skill exists).
- Options that conflict are disabled with a reason tooltip; choosing one that requires another selects it automatically and says so.
- State lives in the URL hash, so any configuration is a shareable link; no account and no server.
- Accessible by keyboard; works at phone width.
- Acceptance: a reload or a pasted link restores the exact selection; no invalid combination can produce a command.

F2. Presets

One-click starting points such as SaaS starter, API only (FastAPI), Web + mobile, and Indian SaaS (Razorpay). A preset is just a saved `stack.json` that CI builds weekly. Acceptance: every preset is Proven or labelled otherwise.

F3. Conflict and dependency engine

Rules live in `registry.json` as `requires`, `excludes`, `filters` and `defaults`, evaluated identically by the UI and the CLI. Each rule has an ID, a user-facing message and a test combination. Acceptance: the UI and the CLI reject the same invalid configurations; a rule exists only with a test that fails without it.

F4. Advanced accordion

A collapsed section with one textbox and quick-add chips (realtime, search, AI features, i18n, infrastructure as code, multi-tenant, admin dashboard). The text is saved as `notes` in `stack.json` and written under "Additional requirements" in `AGENTS.md`.

- Treated as plain data: never run in a shell, never put in a URL path or query beyond the hash payload, length capped at 2,000 characters.
- The prompt tells the agent to implement the notes on top of the selected stack, to ask when they conflict with selections, and never to weaken the security baseline.

F5. Stack spec (`stack.json`)

The single source of truth: selected option IDs, target agents, preset ID, registry version and notes. It is committed to the generated repo so the project can be regenerated or extended later (`stackskills add`).

F6. CLI (`npx stackskills`)

- `init`: reads the config, shows what will be installed and asks consent (or `--yes` for CI), installs skills, writes files.
- `add`: adds an option to an existing repo and updates `stack.json` and `AGENTS.md`.
- `doctor`: checks the repo against the verification gate and reports drift.
- Wraps the open `npx skills add` CLI so installs land in the correct folder for each agent; with `--agent` flags for Claude Code, Codex and Cursor.
- Pins every skill to a commit SHA from the registry; `update` bumps pins after CI passes.
- Telemetry is off by default, and the wrapper sets the underlying skills CLI's telemetry opt-out.

F7. Skill registry (`registry.json`)

A versioned file in git that maps every option to its skills, source repo, license, pinned SHA, supported agents, required tools and test status. It drives the UI, the CLI and the CI matrix. Adding an option = one PR.

F8. Base skills (we write these)

- `stackskills-architect`: monorepo layout, environment validation, error handling, logging, API conventions, how to combine the chosen option skills.

- `stackskills-security`: the always-on security baseline (section 6).
- `stackskills-verify`: the verification gate and the final report format.
- `stackskills-integrations`: short notes for combinations that break (for example Clerk with Fastify, Stripe webhooks on serverless). Added only when CI finds a failure.

F9. Generated instruction files

`AGENTS.md` is canonical (readable by Codex, Cursor and others). `CLAUDE.md` is a one-line pointer to it. `PROMPT.md` holds the build brief. Skills are installed per agent because Claude Code reads `.claude/skills/` while Codex and others read `.agents/skills/`.

F10. Build prompt

The prompt tells the agent to: read `stack.json`; scaffold with each framework's official CLI (not hand-written boilerplate); apply the matching skills; implement a minimal working vertical slice (auth, one database table, one protected route, one payment or email flow if selected); then run the verification gate and fix failures before stopping.

F11. Code graph (default on, opt-out)

The CLI offers to install graphify (`uv tool install graphifyy` then `graphify install`) and the agent runs `/graphify .` after scaffolding so later sessions query a graph instead of grepping. It needs Python 3.10 or newer; the CLI checks for it, explains what graphify modifies (skill files, hooks) and skips cleanly if declined. Code is parsed locally; optional semantic extraction of docs needs an API key.

F12. Security baseline (always on)

Detailed in section 6. Not selectable; the UI shows it as a locked, checked section so users know it is applied.

F13. Verification gate and weekly CI matrix

- Gate inside every generated repo: install, typecheck, lint, unit tests, build, Docker build, health endpoint responds, secret scan clean, dependency audit clean, SAST clean.
- Weekly CI in the StackSkills repo generates each preset and the top 20 combinations with a headless agent run, executes the gate and publishes a public pass/fail matrix. A failing combination is downgraded from Proven automatically.
- Acceptance: the UI badge for any option always reflects the last CI result.

5. Option catalog and conflict rules

Twenty sections. Security, Docker, lint/format, health endpoint, `.env.example` with validated env schema, AGENTS.md and the verification gate are always on and not shown as options.

#	Section	Options
1	App type	Web app + API; Full-stack single app; API only; Web + mobile

#	Section	Options
2	Package manager and monorepo	bun + Turborepo; pnpm + Turborepo; npm workspaces; uv (Python)
3	Frontend	Next.js; Vite + React; SvelteKit; Nuxt; None
4	UI	Tailwind + shadcn/ui; Tailwind only; Component library (MUI/Mantine); Plain CSS
5	Backend	Hono; Express; Fastify; FastAPI; Next route handlers; None
6	Runtime	Node; Bun; Python; Cloudflare Workers
7	Database	Postgres; MySQL; SQLite; MongoDB; None
8	DB provider	Neon; Supabase; Railway/Render managed; AWS RDS; Local Docker
9	ORM	Drizzle; Prisma; SQLAlchemy; Mongoose
10	Auth	Better Auth; Clerk; Firebase Auth; Auth.js; None
11	Payments	Stripe; Razorpay; Lemon Squeezy / Paddle; None
12	Cache and queues	Redis + BullMQ/Celery; Upstash Redis; Postgres jobs (pg-boss); None
13	File storage	AWS S3; Cloudflare R2; Supabase Storage; None
14	Email	Resend; AWS SES; Postmark; None
15	Monitoring	Sentry; Sentry + PostHog; OpenTelemetry; None
16	Testing	Vitest + Playwright; Jest + Cypress; pytest; Minimal
17	CI/CD	GitHub Actions; GitLab CI; None
18	Cloud / deploy	Railway; Render; AWS; Azure; Cloudflare; Vercel + separate backend
19	AI tools used	Claude Code; Codex; Cursor (multi-select)
20	Advanced	Free-text accordion (F4)

Conflict rules (v1, enforced in UI and CLI)

ID	Rule
R1	FastAPI sets runtime to Python, ORM to SQLAlchemy, package manager to uv, testing to pytest; hides Drizzle, Prisma, Mongoose.

ID	Rule
R2	TypeScript backends hide FastAPI and SQLAlchemy.
R3	Cloudflare Workers hides Express, Fastify, BullMQ, Prisma (without edge setup); Redis becomes Upstash.
R4	MongoDB forces Mongoose and hides Neon and RDS.
R5	SQLite hides Neon and RDS (local or edge providers only).
R6	Supabase provider and Supabase Storage both require Postgres and each other.
R7	Better Auth and Auth.js require a database (not None).
R8	Any payment provider requires a database.
R9	Clerk requires a JavaScript frontend (not None).
R10	Full-stack single app hides separate backend options; Next route handlers require Next.js.
R11	Cloudflare deploy allows only Workers-compatible choices.
R12	Vercel hosts the frontend only; the backend must be Railway, Render, AWS or Azure unless Next route handlers are used.
R13	Razorpay shows an INR-only note; subscriptions use Razorpay Subscriptions.
R14	BullMQ requires Redis and Node or Bun; excluded on Workers and serverless-only hosts.

Options outside the 20 sections (realtime, search, AI features, i18n, infrastructure as code, multi-tenant, SMS and push) go through the Advanced box until they have enough demand to become sections.

6. Security baseline (always on)

Security is applied to every generated repo and cannot be unchecked. It has three layers: skills that steer the agent, controls written into the code, and scanners that must pass in the verification gate.

Honest scoping. I did not find a single published "Cloudflare security audit" skill. Cloudflare's official skills repo has a broad platform skill that covers Workers, storage and security products such as WAF and DDoS, so StackSkills installs it for every project and its own `stackskills-security` skill adds the audit checklist below. This skill is ours to write.

Layer	Control	How it is applied
Edge	WAF, DDoS, bot protection, Turnstile on public forms	Cloudflare skill plus a `NEXT_STEPS.md` checklist (DNS and dashboard steps cannot be automated by the agent)

Layer	Control	How it is applied
App	Security headers and CSP, strict CORS, input validation on every route, rate limiting on auth and payment routes	`stackskills-security` skill, verified by tests in the gate
Auth	Secure session cookies, CSRF protection, no secrets in client bundles, least-privilege roles	Provider skill (Clerk, Better Auth, Firebase) plus `stackskills-security`
Payments	Webhook signature verification and idempotency keys, no card data stored	Stripe or Razorpay skill plus a required webhook test
Data	Parameterized queries via ORM, row-level security where the provider supports it, least-privilege DB user, migrations reviewed	Postgres and ORM skills
Secrets	Validated env schema, `.env` ignored, `.env.example` only, secret scanner in CI	Gate step: Gitleaks
Supply chain	Lockfile committed, dependency audit, pinned GitHub Actions, Dependabot or Renovate	Gate step: OSV-Scanner, `npm audit` or `pip-audit`
Code	Static analysis of generated code	Gate step: Semgrep (open-source rules) in CI; optional Semgrep Guardian plugin for live scanning during agent edits
Containers	Non-root user, pinned base image, minimal image, scan	Docker skill plus Trivy in the gate
Agent safety	Skills and prompts treated as untrusted input; no auto-run of unknown scripts	Consent prompts in the CLI, SHA pinning, review of every skill before it enters the registry

Periodic audit. The prompt ends with a security review pass. For Claude Code, the `claude-security` plugin from Anthropic's official plugin marketplace is reported to run a deep repository audit; treat that as an optional extra step, since a community guide notes version-compatibility problems with some Claude Code releases.

Acceptance: a generated repo fails the gate if any of these is missing or if a scanner reports a high-severity finding.

7. Skills catalog: use directly, vet, or write

Found by search on 2026-10-25. Vendor-maintained skills are the safest to use directly. Community skills must be reviewed (and pinned to a commit) before they enter the registry. Licenses shown are only

those I saw stated; check each repo's LICENSE before vendoring. Skills are installed with the open skills CLI following the Agent Skills spec.

A. Vendor-maintained skills (use directly)

Option	Source	Install
Next.js	vercel-labs/next-skills	`npx skills add vercel-labs/next-skills`
React / Next.js performance	vercel-labs/agent-skills (MIT stated in README)	`npx skills add vercel-labs/agent-skills`
Expo / React Native	expo/skills (MIT)	`npx skills add expo/skills`
shadcn/ui	ui.shadcn.com skills	`npx skills add shadcn/ui`
Postgres (any host)	supabase/agent-skills (MIT)	`npx skills add supabase/agent-skills`
Neon	neondatabase/agent-skills	`npx skills add neondatabase/agent-skills -s neon-postgres`
Prisma	prisma/skills	`npx skills add prisma/skills`
Better Auth	better-auth/skills	`npx skills add better-auth/skills`
Clerk	clerk/skills, index at clerk.com	`see Clerk's skills index (command not confirmed)`
Firebase (auth, Firestore, hosting)	firebase/agent-skills (Apache 2)	`npx skills add firebase/agent-skills`
Stripe	stripe/ai, docs at docs.stripe.com/skills	`npx skills add https://github.com/stripe/ai`
Redis	redis/agent-skills (MIT)	`npx skills add redis/agent-skills`
Upstash (Redis, QStash, rate limit)	upstash/skills	`npx skills add upstash/skills`
Resend + React Email	resend/resend-skills	`npx skills add resend/resend-skills`
Sentry	getsentry/sentry-for-ai (MIT)	`npx skills add getsentry/sentry-for-ai`
PostHog	PostHog/ai-plugin	see repo README

Option	Source	Install
Playwright	microsoft/playwright-cli	`npx skills add https://github.com/microsoft/playwright-cli --skill playwright-cli`
Docker	docker/skills, docs	see docs
Railway	railwayapp/railway-skills (MIT)	`npx skills add railwayapp/railway-skills`
Render	render-oss/skills (MIT)	plugin or install script in README
AWS	awslabs/agent-plugins (README says Agent Toolkit for AWS is the successor; re-check before pinning)	Claude Code: `/plugin marketplace add awslabs/agent-plugins`
Azure	microsoft/azure-skills	`npx skills add microsoft/azure-skills --skill azure-deploy`
Cloudflare (Workers, D1, R2, WAF, DDoS)	cloudflare/skills	`npx skills add https://github.com/cloudflare/skills`
Code graph	Graphify-Labs/Graphify (MIT)	`uv tool install graphifyy` then `graphify install`
Skill authoring	anthropics/skills: example skills Apache 2.0; document skills source-available, not open source	copy `skill-creator`

B. Community skills (vet first, then pin)

Option	Candidate	Note
Hono	yusukebe/ai-with-hono	Wraps the Hono CLI for docs and request testing
Drizzle	OiAnthony/drizzle-orm-skill	Describes itself as independent community, not official
FastAPI	KaribuLab/skills (python-fastapi), MUmerRazzaq/fastapi-rest-api-skill	Small, low-adoption; likely rewrite
SQLAlchemy, Pydantic	bobmatnyc/claude-mpm-skills	Large third-party collection; extract only after review

Option	Candidate	Note
Security audit	trailofbits/skills	Licensed CC BY-SA 4.0: reference or install from source, do not copy into our repo without share-alike
Live SAST during edits	Semgrep Guardian	Setup requires a Semgrep account; keep optional

C. Gaps: we write these (nothing suitable found)

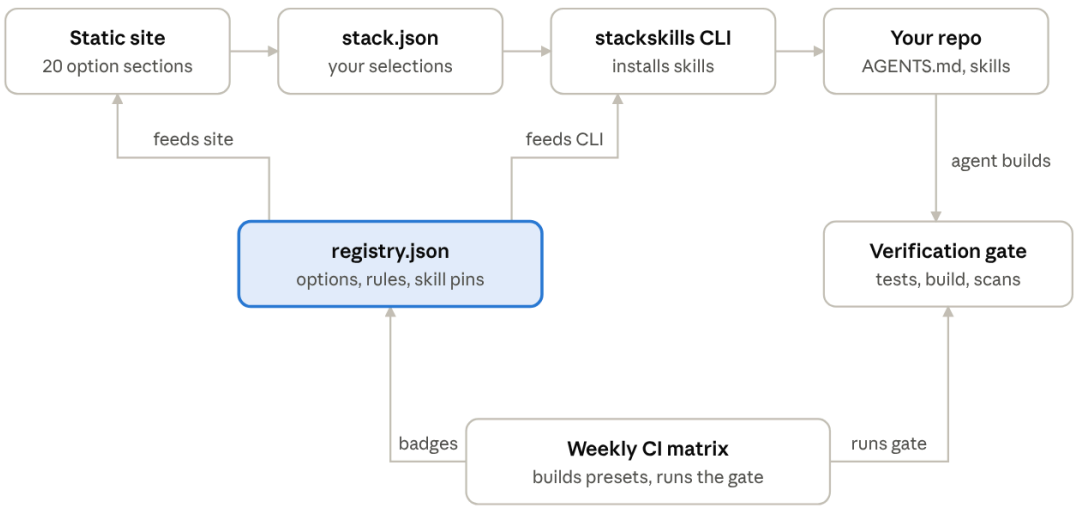
- Razorpay: only an MCP server exists (docs); write `razorpay-payments` covering orders, signature verification, webhooks, subscriptions and INR specifics.
- Express, Fastify, NestJS conventions.
- Tailwind (general, non-Expo), Vitest, pytest conventions.
- Auth.js, Lemon Squeezy, Paddle, Mongoose/MongoDB, SES, Postmark, GitLab CI, S3 storage patterns: not found or not researched; check again before building these, and mark options Experimental until a skill exists.
- Our base skills: `stackskills-architect`, `stackskills-security`, `stackskills-verify`, `stackskills-integrations` (section 4, F8).

Before each option ships, run the vendor's skill on a clean repo, read its SKILL.md and any scripts, and record the reviewed commit SHA in `registry.json`.

8. Technical architecture

StackSkills is two static artifacts, a site and a CLI, both driven by one registry file; there is no server and no database.

One registry drives the site, the CLI and the weekly CI



architecture · 3 apps, 1 registry

The registry feeds the site and the CLI. The CLI installs pinned skills into the user's repo, an agent builds the project, and the same verification gate that ships inside the repo is run by the weekly CI, whose results become the badges in the registry.

Monorepo layout

- `apps/web`: the builder site (Vite + React + Tailwind, static output).
- `packages/core`: pure functions that load the registry, evaluate rules and render `AGENTS.md` and the prompt; shared by the site and the CLI so they can never disagree.
- `packages/cli`: the `stackskills` npm package; wraps `npx skills add`.
- `registry/registry.json` and `registry/rules.json`: options, rules, skill sources and pinned commits.
- `skills/`: our own base skills (F8).
- `presets/`: saved `stack.json` files, each built weekly.
- `ci/`: the matrix runner and the gate script.

Hosting and cost. Site on a free static host. Users pay only for their own agent usage. The weekly CI runs headless agents, which costs the maintainer real tokens; limit it to presets and the top combinations, cache installs, and run on schedule rather than on every commit.

`stack.json` (example)

```
{
  "registryVersion": "2026.10.1",
  "preset": "saas-starter",
  "agents": ["claude-code", "codex", "cursor"],
  "choices": {
    "frontend": "nextjs",
    "backend": "hono",
    "runtime": "node",
    "database": "postgres",
    "dbProvider": "neon",
    "orm": "drizzle",
    "auth": "better-auth",
    "payments": "stripe",
    "deploy": "railway"
  },
  "notes": "Add an admin dashboard."
}
```

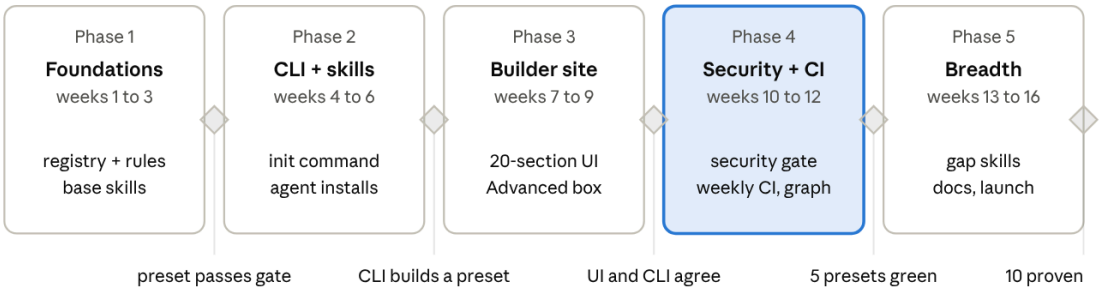
Registry entry (example)

```
{
  "id": "auth.better-auth",
  "label": "Better Auth",
  "skills": [{ "source": "better-auth/skills", "sha": "<reviewed commit>", "license":
"<from LICENSE>" }],
  "requires": ["database != none"],
  "excludes": [],
  "status": "experimental",
  "lastCi": null
}
```

9. Roadmap: five phases

Sixteen weeks to a public launch, assuming one full-time developer; the weeks are my estimates, not measured. Part-time work roughly doubles them. Phase 4 holds the security work because it must land before any preset is called Proven.

Five phases, each ending in a gate that must pass



Diamond: a gate. The next phase starts only after it passes. Bands are equal, not to scale.

roadmap · 5 phases, 5 gates

Phase	Weeks (est.)	Scope	Exit gate
1. Foundations	1 to 3	Monorepo; registry schema; rules engine in `packages/core`; `stack.json` spec; first 3 presets (SaaS starter, FastAPI API, web + mobile) hand-built as references; draft base skills (`stackskills-architect`, `stackskills-verify`); gate script	One reference preset passes the full gate by hand
2. CLI + skills	4 to 6	`stackskills init`; consent prompts; wrapper over `npx skills add` for Claude Code, Codex, Cursor; SHA pinning; generation of `AGENTS.md`, `CLAUDE.md`, `PROMPT.md`; telemetry opt-out; first vendor skills vetted (Next.js, Postgres, Neon, Better Auth, Stripe, Railway)	An agent builds one preset from an empty folder and the gate passes
3. Builder site	7 to 9	20-section UI; conflict engine from `core`; Advanced accordion; URL-hash state; presets; summary and command box; keyboard and phone layout	Any selection restores from a link; UI and CLI reject the same invalid configs

Phase	Weeks (est.)	Scope	Exit gate
			(shared test suite)
4. Security + CI	10 to 12	`stackskills-security` skill; Cloudflare skill wired in; gate adds Gitleaks, OSV-Scanner, Semgrep, Trivy, dependency audit; graphify opt-out install; weekly CI matrix with a public pass/fail page; badges in the UI	5 presets pass weekly CI with Claude Code and Codex; Cursor checked manually unless headless CI support is confirmed
5. Breadth + launch	13 to 16	Write gap skills (Razorpay, Express, Fastify, FastAPI rewrite, Tailwind, Vitest, pytest); `add` and `doctor` commands; contributor guide (add an option in one PR); license audit of every skill; docs site; launch post	10 presets Proven; 80% of options have a vetted skill; every skill's license recorded

Order of value if time runs short. Phases 1, 2 and 4 deliver a working, secure command-line product; Phase 3 makes it easy to choose; Phase 5 widens coverage. A command-only release after Phase 2 plus the security gate from Phase 4 is a viable early version.

10. Risks, open questions, licensing

Risk	Why it matters	Mitigation
Agent output varies between runs	A skills-first build is not deterministic	Scaffold with official CLIs; `stack.json` spec; verification gate; public CI matrix; downgrade failing combinations
Supply-chain attack through a skill	Skills steer an agent that has shell access; some include scripts	Pin SHAs; review each skill and script; show the list and ask consent; never auto-run unknown scripts; CI scans the registry
Skill coverage gaps	No skill found for Razorpay, Express, Fastify, Tailwind (general), Vitest, pytest	Mark options Experimental until a skill exists; write the skills in Phase 5; accept community PRs
Agent compatibility drift	Install paths and frontmatter support differ and change between releases; reports of Cursor not loading some skill assets	Install per agent; test per agent in CI; keep `AGENTS.md` as the canonical file; re-verify paths each release

Risk	Why it matters	Mitigation
Maintenance load	Vendor skills and framework versions change weekly	Pinned versions plus a scheduled bump PR that must pass CI; narrow the option list before widening it
CI token cost	Headless agent runs cost real money	Presets and top combinations only; scheduled, not per commit; cache installs
Crowded space	Better-T-Stack already offers a builder, a CLI, a Claude Code plugin and a skills add-on	Differentiate on breadth (Python, Razorpay, more clouds), the always-on security gate and the public matrix
Graphify requirement	Needs Python 3.10 or newer and modifies agent configuration	Opt-out, explain what it changes, skip cleanly when Python is missing

Licensing. The StackSkills code is MIT. Third-party skills keep their own licenses and are referenced by URL and pinned SHA, not copied, unless the license allows it. Trail of Bits skills are CC BY-SA 4.0 (share-alike), so reference them rather than vendoring. Anthropic's document skills are source-available, not open source. Record each skill's license in `registry.json` and surface it in the UI before install.

Open questions

- ☐ Final product name and npm package name (check availability).
- ☐ Which 3 presets ship first, and is Python in the first three?
- ☐ Is Cursor testable headlessly in CI, or checked manually?
- ☐ Who funds the weekly CI token cost (maintainer, sponsor, or a reduced schedule)?
- ☐ Clerk, Drizzle, SQLAlchemy and Hono skills: official versus community, and which to pin.
- ☐ Should graphify stay default-on once its Python requirement is clear to users?